

个人作业报告

题目：**并行程序设计与算法**

理论课第三次作业

姓名	林宏宇
学号	23320093
院系	计算机学院
专业	计算机科学与技术
指导教师	吴迪

2026 年 4 月 29 日

【摘　要】

此作业报告是并行程序设计与算法课程的**第三次作业**，作业题目来自智算习堂[1]的作业内容。

关键词：　并行程序设计与算法

目录

1 第一题 1
1.1 题目 1
1.2 答案 2

2 第二题 4
2.1 题目 4
2.2 答案 5

3 第三题 6
3.1 题目 6
3.2 答案 6

4 第四题 8
4.1 题目 8
4.2 答案 9

5 第五题 11
5.1 题目 11
5.2 答案 12

参考文献 14

1 第一题

1.1 题目

考虑以下两个计算 1 到 N 之间所有整数平方和的 Pthreads 程序片段。假设 N= 10^9，线程数 thread_count= 8

代码段 A：

```
void* Square_Sum_A(void* rank){
    long my_rank=(long) rank;
    long long local_n= N/ thread_count;
    long long first= my_rank* local_n+ 1;
    long long last= first+ local_n;
    for(long long i= first; i< last; i++){
        pthread_mutex_lock(&mutex);
        global_sum+= i* i;
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

代码段 B：

```
void* Square_Sum_B(void* rank){
    long my_rank=(long) rank;
    long long local_n= N/ thread_count;
    long long my_sum= 0;
    long long first= my_rank* local_n+ 1;
    long long last= first+ local_n;
    for(long long i= first; i< last; i++){
        my_sum+= i* i;
    }
    pthread_mutex_lock(&mutex);
    global_sum+= my_sum;
    pthread_mutex_unlock(&mutex);
    return NULL;
}
```

回答下列问题：

1) 指出代码段 A 中的临界区，并估算在 N=10^9 时，代码段 A 总共执行了多少次锁操作
2) 从操作系统调度的角度分析，为什么代码段 A 会导致 CPU 利用率虚高但实际计算进度缓慢
3) 在代码段 B 中，主线程在创建线程后立即调用 pthread_join（阻塞调用它的线程，直到指定的线程结束），请说明 pthread_join 在此处的作用

1.2 答案

1.2.1 代码段 A 中的临界区及锁操作次数

代码段 A 中的临界区是包含全局变量 global_sum 更新的代码块：

```
pthread_mutex_lock(&mutex);
global_sum+= i* i;
pthread_mutex_unlock(&mutex);
```

当 N=10^9，thread_count=8 时，每个线程处理的元素数量为 local_n = 10^9/8 = 125,000,000（整数除法）。因此每个线程执行 125,000,000 次循环，每次循环执行一次加锁和解锁操作，共 250,000,000 次锁操作（125,000,000 次加锁 +125,000,000 次解锁）。8 个线程总共执行 8×250,000,000=2,000,000,000 次锁操作。

1.2.2 CPU 利用率虚高但计算进度缓慢的原因

代码段 A 在每次累加操作前都进行加锁和解锁，导致频繁的上下文切换和线程阻塞。从操作系统调度角度看：

- 当一个线程持有锁时，其他试图获取锁的线程会被阻塞并进入等待队列
- 操作系统需要频繁进行线程调度和上下文切换，消耗大量 CPU 时间
- 实际用于计算的时间比例很小，大部分时间花在锁竞争和调度上
- 虽然 CPU 显示高利用率（因为频繁调度），但有效计算工作很少

1.2.3 pthread_join 的作用

在代码段 B 中，pthread_join 的作用是：

- 阻塞主线程，直到指定的工作线程完成执行
- 确保所有工作线程都完成计算并将局部结果累加到 global_sum 后，主线程才

能继续执行

- 保证程序正确性，避免主线程在工作线程完成前就结束或访问未完成的结果
- 提供线程同步机制，确保所有并行计算任务完成后再进行后续操作

2 第二题

2.1 题目

读写锁允许多个读者同时进入临界区，但写者必须互斥访问。考虑以下一个简单的、基于信号量实现的自定义读写锁逻辑：

```
int read_count= 0;
sem_t count_mutex;//保护 read_count的更新，初始化为 1
sem_t rw_mutex; //保护共享资源的写操作，初始化为 1

void* reader(void* arg){
    sem_wait(&count_mutex);
    read_count++;
    if(read_count== 1) sem_wait(&rw_mutex);//第一个读者锁定资源
    sem_post(&count_mutex);
    //执行读操作
    sem_wait(&count_mutex);
    read_count--;
    if(read_count== 0) sem_post(&rw_mutex);//最后一个读者释放资源
    sem_post(&count_mutex);
    return NULL;
}

void* writer(void* arg){
    sem_wait(&rw_mutex);
    //执行写操作
    sem_post(&rw_mutex);
    return NULL;
}
```

回答下列问题：

1) 描述在该实现下，什么情况下写者可能会遭遇” 饥饿” 现象？
2) 该实现倾向于” 读者” 还是” 写者”？ 如何避免对写者的饥饿问题，简要描述你的方案

2.2 答案

2.2.1 写者饥饿的情况

在该实现下，写者可能会遭遇饥饿现象的情况是：

- 当系统中有持续不断的读者请求到达时
- 只要有至少一个读者在临界区内，rw_mutex 就会被占用
- 新的读者可以不断进入（因为 read_count>0 时不需要获取 rw_mutex）
- 写者需要等待所有读者都离开（read_count=0）才能获取 rw_mutex
- 如果读者流持续不断，写者可能永远无法获得执行机会

2.2.2 倾向性及避免饥饿的方案

该实现明显倾向于” 读者”，因为：

- 多个读者可以并发执行
- 读者不需要等待其他读者
- 写者必须等待所有读者完成

避免写者饥饿的方案：

方案一：写者优先策略

- 引入写者等待计数器 write_waiting
- 当有写者在等待时，阻止新读者进入
- 修改读者代码，在获取 count_mutex 后检查是否有写者等待
- 如果有写者等待，则读者需要等待直到写者完成

方案二：公平队列策略

- 使用 FIFO 队列管理所有请求（读者和写者）
- 按照请求到达顺序依次处理
- 允许连续的读者请求一起处理以提高效率
- 但确保写者不会被无限期推迟

3 第三题

3.1 题目

某多线程程序中有两个共享数据结构 A 和 B，分别由互斥锁 mutexA 和 mutexB 保护。程序启动后有两个线程 T0 和 T1 并发执行，它们的加锁顺序如下：

- 线程 T0：先执行 pthread_mutex_lock(&mutexA)，再执行 pthread_mutex_lock(&mutexB)
- 线程 T1：先执行 pthread_mutex_lock(&mutexB)，再执行 pthread_mutex_lock(&mutexA)

假设系统中出现如下执行时序：

- 时刻 0：T0 成功获得 mutexA，T1 成功获得 mutexB
- 时刻 1：T0 试图获得 mutexB，T1 试图获得 mutexA

请回答下列问题：

1) 此时程序会发生什么现象？请说明原因。
2) 为什么这种问题在线程程序中比较危险？
3) 给出一种简单可行的修改方法，避免该问题再次发生。

3.2 答案

3.2.1 程序现象及原因

此时程序会发生**死锁**（Deadlock）现象。

原因分析：

- T0 持有 mutexA，等待 mutexB
- T1 持有 mutexB，等待 mutexA
- 两个线程都在等待对方释放已持有的锁
- 形成循环等待条件，且没有线程能够继续执行
- 根据死锁的四个必要条件（互斥、占有并等待、非抢占、循环等待），此处全部满足

3.2.2 死锁的危险性

死锁在线程程序中比较危险的原因：

- **程序完全停滞**：涉及死锁的线程永远无法继续执行

- **资源浪费**：占用的系统资源（内存、CPU 时间片等）无法释放
- **难以检测**：死锁可能在特定条件下才出现，具有偶发性
- **影响整个系统**：如果主线程或其他关键线程被卷入，可能导致整个程序崩溃
- **调试困难**：死锁通常没有明显的错误信息，需要专门的工具分析

3.2.3 避免死锁的修改方法

方法一：统一加锁顺序

- 规定所有线程必须按照相同的顺序获取锁
- 例如：总是先获取 mutexA，再获取 mutexB
- 这样可以破坏循环等待条件

方法二：一次性获取所有需要的锁

- 设计函数一次性申请所有需要的锁
- 如果无法同时获取所有锁，则全部释放并重试
- 减少分步加锁的机会

4 第四题

4.1 题目

阅读以下代码片段，回答问题

```c
#include<pthread.h>
#include<stdio.h>
int counter= 0;//全局计数器

void* UnsafeIncrement(void* arg){
    for(int i= 0; i< 100000; i++){
        counter++;
    }
    return NULL;
}

int main(){
    pthread_t t1, t2;
    pthread_create(&t1, NULL, UnsafeIncrement, NULL);
    pthread_create(&t2, NULL, UnsafeIncrement, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    printf("Final Counter Value:%d\n", counter);
    return 0;
}
```

1) 理论上，如果程序正确运行，counter 的最终值应该是多少？为什么实际运行结果通常小于这个理论值?
2) 请指出 counter++ 操作在汇编层面通常包含哪三个步骤？并解释这如何导致数据竞争。
3) 如何修改 UnsafeIncrement 函数，使其线程安全

4.2 答案

4.2.1 理论值与实际值差异

理论上，如果程序正确运行，counter 的最终值应该是 200,000（100,000 × 2）。

实际运行结果通常小于理论值的原因：

- counter++ 操作不是原子操作
- 两个线程同时读取、修改、写入 counter 变量
- 发生数据竞争（Data Race），导致某些增量操作丢失
- 例如：两个线程同时读取 counter=100，各自加 1 后都写回 101，实际只增加了 1 而不是 2

4.2.2 counter++ 的汇编步骤及数据竞争

counter++ 操作在汇编层面通常包含三个步骤：

1) **Load**：从内存加载 counter 的当前值到寄存器
2) **Increment**：在寄存器中对值加 1
3) **Store**：将寄存器中的新值存储回内存中的 counter

数据竞争的产生过程：

- 线程 1 执行 Load，读取 counter=100
- 线程 2 执行 Load，也读取 counter=100
- 线程 1 执行 Increment，寄存器值变为 101
- 线程 2 执行 Increment，寄存器值也变为 101
- 线程 1 执行 Store，counter=101
- 线程 2 执行 Store，counter=101（覆盖了线程 1 的结果）
- 最终结果为 101，而不是期望的 102

4.2.3 线程安全的修改方案

修改方案：使用互斥锁

```
pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;

void* SafeIncrement(void* arg){
    for(int i= 0; i< 100000; i++){
        pthread_mutex_lock(&mutex);
        counter++;
```

```
        pthread_mutex_unlock(&mutex);
    }
    return NULL;
}
```

5 第五题

5.1 题目

假设我们有一个多线程程序，用于统计两类事件发生的次数。两个线程分别只更新自己对应的计数器

```
#include<pthread.h>
#include<stdio.h>

//定义一个结构体，包含两个计数器
typedef struct{
    long counter_a;//线程 0专门累加这个
    long counter_b;//线程 1专门累加这个
} CounterPair;

CounterPair counters;

void* ThreadFunc(void* arg){
    long tid=(long)arg;
    //模拟大量工作
    for(int i= 0; i< 100000000; i++){
        if(tid== 0){
            counters.counter_a++;//线程 0操作
        } else{
            counters.counter_b++;//线程 1操作
        }
    }
    return NULL;
}

int main(){
    pthread_t t0, t1;
    //创建两个线程
    pthread_create(&t0, NULL, ThreadFunc,(void*)0);
    pthread_create(&t1, NULL, ThreadFunc,(void*)1);
```

```
    pthread_join(t0, NULL);
    pthread_join(t1, NULL);
    printf("A:%ld, B:%ld\n", counters.counter_a, counters.counter_b);
    return 0;
}
```

1) 为什么上述代码可能存在严重的伪共享问题，并解释一下为何在两个线程上操作不同的变量，却会导致性能急剧下降?
2) 为了消除伪共享,请修改 CounterPair 结构体的定义,利用填充技术确保 counter_a 和 counter_b 位于不同的缓存行中。假设系统的缓存行大小为 64 字节。
3) 除了填充结构体外，还有另一种利用局部性原理的高性能方案，如何修改 ThreadFunc 函数的逻辑，让每个线程先在" 私有" 的内存空间进行累加，最后再合并结果，从而彻底避开伪共享?

5.2 答案

5.2.1 伪共享问题及性能下降原因

伪共享（False Sharing）问题的存在原因:

- 现代 CPU 的缓存以缓存行（Cache Line）为单位进行管理，通常为 64 字节
- counter_a 和 counter_b 在内存中相邻存储，很可能位于同一个缓存行中
- 虽然两个线程操作不同的变量，但这些变量共享同一个缓存行
- 当线程 0 修改 counter_a 时，会使得整个缓存行在其他 CPU 核心上失效
- 线程 1 需要重新从内存或其他核心获取包含 counter_b 的缓存行
- 频繁的缓存行无效化和重新获取导致大量内存总线流量
- 性能急剧下降，因为实际的内存访问延迟远高于缓存访问

5.2.2 使用填充技术消除伪共享

修改 CounterPair 结构体定义，确保两个计数器位于不同缓存行:

```
#define CACHE_LINE_SIZE 64

typedef struct __attribute__((aligned(CACHE_LINE_SIZE))) {
    long value;
    char padding[CACHE_LINE_SIZE - sizeof(long)];
} Counter;
```

```
7
8  typedef struct {
9      Counter a;
10     Counter b;
11 } CounterPair;
```

这样，counter_a 占据第一个缓存行，counter_b 占据第二个缓存行，完全避免了伪共享。

5.2.3 局部累加方案

修改 ThreadFunc 函数，使用线程局部变量进行累加：

```
1  void* ThreadFunc(void* arg){
2      long tid=(long)arg;
3      long local_counter = 0; // 线程局部计数器
4
5      //模拟大量工作
6      for(int i= 0; i< 100000000; i++){
7          local_counter++; // 在私有内存中累加
8      }
9
10     // 最后一次性更新全局计数器
11     if(tid == 0){
12         counters.counter_a = local_counter;
13     } else{
14         counters.counter_b = local_counter;
15     }
16
17     return NULL;
18 }
```

这种方案的优势：

- 完全避免了伪共享，因为累加操作在栈上的局部变量进行
- 只有一次全局内存写入，大大减少缓存行争用
- 性能显著提升，特别是在线程数较多的情况下
- 代码简洁，易于理解和维护

参考文献

[1] [Z]. https://easyai.sysu.edu.cn.